COMP-8117 · SPLASH PROJECT PLAN · OPTION B

"No Food Left Behind"

A simple pickup and stock system for a campus food bank, built the software engineering way

Team: *[names]* **Duration:** 10 weeks **Defence:** December 2026 **Version:** 1.0

01 Problem

Campus food banks serve students who are struggling, often international students. Most are run by volunteers using paper sign-in sheets and spreadsheets. This causes two problems:

- **Students wait in long lines** without knowing what's in stock, and sometimes leave with nothing.
- **Food expires on the shelf** because nobody can easily see what expires first or what's running out.

> **Problem statement.** How can a volunteer-run food bank let students book a pickup in advance and see what's available, while volunteers track stock and expiry dates, with software simple enough that new volunteers can use it with no training?

Impact. Shorter waits and more dignity for students. Less food wasted and less admin for volunteers. The software is small, but the people it helps are real. *Partner: confirm a real campus food bank (e.g. the student union's) in week 2–3.*

02 Why this is a software engineering project

IEEE defines software engineering as "*the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software.*" The product is deliberately simple, so we can put the effort into doing each part of that definition properly:

Principle	What it means in our project
Systematic	We follow the full lifecycle: requirements from real users → design → build → test → deploy → hand over for maintenance
Disciplined	User stories with acceptance criteria, code review on every change, automated tests in CI, dev → staging → prod
Quantifiable	We measure everything: wait time, wasted items, task completion in usability tests, test coverage, defects found
Operation + maintenance	We deliver docs and a handover so volunteers can keep running it after we leave

03 Users and requirements

Requirements come from interviewing **3–5 volunteers and 5–10 students** (week 3), not from guessing.

User	Key user stories (draft)
Student	See what's in stock today · Book a 15-minute pickup slot · Get a reminder · Cancel if I can't come
Volunteer	Add donated items with an expiry date in under 30 seconds · See "expiring this week" first · Check students in at pickup
Coordinator	Weekly report: visits, items given out, items wasted

Non-functional: works well on a phone · usable by a new volunteer with no training · privacy first: we store only what's needed (name + student email), no sensitive data · accessible (WCAG basics).

04 Solution and design

Student web app
Stock list, slot booking, reminders

→

Backend API
Bookings, inventory, check-in, reports

→

Database
Postgres: items, expiry dates, slots, visits

←

Volunteer app
Add stock, expiry alerts, check-in

One simple web app with two views. Standard, well-known tech (e.g. React or Next.js + Node/FastAPI + Postgres), so the next maintainers can understand it. **What we will not build:** payments, a native mobile app, complex analytics.

05 Quality plan

Activity	How	Target
Automated tests	Unit + API tests run in CI on every pull request	≥ 80% coverage of core logic
Code review	Every change reviewed by another team member	100% of merges
Usability testing	5 students + 3 volunteers complete real tasks, twice (week 6 and week 9)	≥ 90% task success
Acceptance	Each user story passes its acceptance criteria in staging	All "must-have" stories
Defect tracking	Bugs logged as issues with severity	0 open critical bugs at handover

06 Measuring impact

Metric	Before (baseline, week 3–4)	After (pilot, week 8–9)
Average student wait at pickup	*measure on site*	target –50%
Items thrown away (expired) per week	*count with volunteers*	target –30%
Volunteer time on admin per shift	*ask / time it*	target –30%

If no real pilot is possible, we run a simulated pickup day with classmates and report those numbers instead.

07 Environments

Dev: each laptop, Docker Compose, fake data. **Staging:** cloud copy where acceptance and usability tests run. **Prod:** the pilot the food bank actually uses, deployed only from releases that passed staging, with daily database backups.

08 Team we need

What everyone does

- Takes part in interviews and usability tests, writes code and tests, reviews pull requests
- Joins the weekly sync, keeps the decisions log, presents their own area at the defence

Roles

Role	Owner	Leads
Platform / DevOps	Abhinav	Repo, CI/CD, environments, backups, deployment, handover docs
Product + UX	*[name]*	Interviews, user stories, screens, usability tests
Backend + QA	*[name]*	API, database, automated tests, defect tracking

09 10-week plan

Week	Lifecycle phase	Deliverable
1–2	Problem + team	Problem statement, team, this plan, food bank contact
3	Requirements	Interviews, user stories with acceptance criteria, baseline measurements
4	Design	Screen sketches, data model, architecture, test plan
5–6	Build v1	Booking + stock list + check-in; CI and staging live; usability test #1
7	Build v2	Expiry alerts, reminders, weekly report; fixes from usability test
8–9	Pilot + test	Prod pilot, usability test #2, impact measurements
10	Handover + defence	Maintenance guide, final numbers, demo, slides

10 Budget and risks

Budget ≈ $40–$60 total: a small hosting plan (App Runner or a free tier) and a small managed Postgres, plus free email reminders and a free CI tier. Capped at $100/month with an AWS Budget alert.

Risk	Response
No real food bank partner	Simulated pickup day with classmates; same metrics
Privacy of student data	Collect the minimum, no sensitive fields, delete pilot data after the project
Scope creep	Only "must-have" stories are committed; the rest go in a backlog
Looks "too simple"	The grade is in the process and evidence: requirements, tests, usability results, measured impact

Definition source: IEEE, as cited in OpenStax, *Introduction to Computer Science*, §9.1 "Software Engineering Fundamentals".